

Project-Bragboard

Name: Lakshmi Prasanna

milestone-2:

Week 3 – Shout-Out Posting & Feed

overview:

In this milestone, the goal is to implement the **core shout-out posting feature** that enables users to appreciate others by tagging and posting recognition messages. This forms the foundation of the interactive social feed within the platform.

1. Shout-Out Form with Recipient Tagging:

A responsive form is designed to allow users to create shout-outs and tag specific recipients. The form includes input fields for the message content, recipient selection (with a dropdown or autocomplete feature), and optional media inputs. Tagging ensures that shout-outs are directed to the right users and that the data is structured for backend mapping.

2. Database Storage with Tagged Users:

When a user submits a shout-out, the data (including sender ID, message, timestamp, and tagged user IDs) is stored in the database. Relationships are set between shout-outs and user tables to easily retrieve who sent and received each message. This setup supports efficient fetching for the upcoming feed display module.

Overall, this task introduces the **posting functionality** and lays the groundwork for the **shout-out feed system**, promoting user engagement and recognition within the application.

Common Errors Faced

1. **Form data not submitting properly** — Bind inputs to state and ensure the submit handler runs (use `e.preventDefault()` and send the payload).
2. **Tagged users not saving in database** — Validate and send an array of tagged user IDs, and have the backend map/save them to the shout-out relation.
3. **Undefined/null error in feed display** — Guard rendering with loading checks or use optional chaining (e.g., `shout?.tags`) to avoid null references.
4. **API connection error** — Verify the endpoint URL, CORS settings, and include the correct auth token in request headers.

Project Setup and Environment Configuration

- A monorepo structure was created, separating the project into a dedicated backend directory for the FastAPI application and a frontend directory for the React.js application.

- The backend environment was configured with a Python virtual environment and all necessary dependencies, including SQLAlchemy for database interactions.
- The frontend environment was initialized using Vite, and Tailwind CSS was installed and configured for efficient styling.
- A scalable folder structure was implemented for both backend and frontend to ensure maintainability:
 - Backend: models, schemas, crud, api
 - Frontend: pages, components, services

Instructions for Running the Project

To run the BragBoard project, both the backend (FastAPI) and frontend (React) need to be set up and running. Follow these steps:

Backend (FastAPI)

1. Open a terminal and navigate to the backend directory: `cd backend`
2. Create and activate a Python virtual environment:
 - Windows: `python -m venv venv → venv\Scripts\activate`
 - macOS/Linux: `python -m venv venv → source venv/bin/activate`
3. Install the required dependencies using `pip install -r requirements.txt`
4. Start the FastAPI server using: `uvicorn app.main:app --reload`
 - The backend will run at <http://127.0.0.1:8000/docs>

Frontend (React with Vite)

1. Open a terminal and navigate to the frontend directory: `cd frontend`
2. Install the required packages using `npm install`
3. Start the development server with `npm run dev`
 - The frontend will run at <http://localhost:5173>

Project Repository

The source code for BragBoard is hosted on GitHub and can be accessed at the following link:

[BragBoard Repository](#).